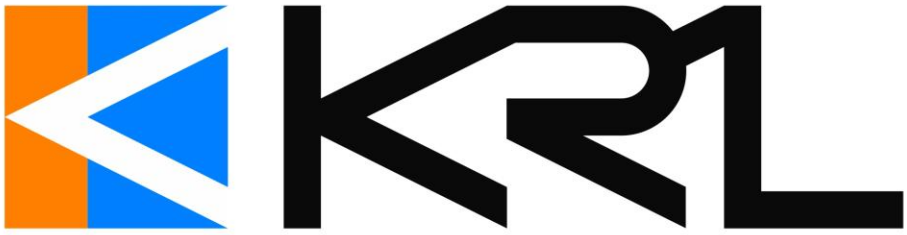

SOFTWARE ENGINEERING

KRL Predictor

Local Refractive Index Calculation System

SE

Table of Contents

Table of Figures

1. Project Information

1.1. Project Identification

System Name	KRL Predictor
Version	1.0
Date	February 2026
Methodology	Adapted Agile Development (Simplified XP)

Table 1: Project Identification

1.2. Development Team

Role	Responsible	Responsibilities
Domain Expert / Product Owner	[Formula Creator]	Scientific validation, requirements definition, KRL formula specification
Developer / Implementer	[Software Engineer]	Software design and implementation, graphical interface, testing, technical documentation

Table 2: Development Team Distribution

2. Development Methodology

2.1. Methodological Approach

Given the context of a small team composed of the scientific formula creator and the technical implementer, a simplified agile approach based on Extreme Programming (XP) has been adopted. This methodology is ideal for small projects where direct communication and continuous feedback are essential.

2.2. Applied Principles

- Direct communication: Constant interaction between the domain expert and the developer to validate the KRL formula implementation.
- Simplicity: Minimalist design focused on the essential functionality of the refraction coefficient calculation.
- Immediate feedback: Continuous scientific validation of results through testing with real data.
- Short iterations: Incremental development with frequent functional deliveries.
- Clean code: Clear and well-documented implementation that facilitates future maintenance.

2.3. Development Cycle

Phase	Activities	Deliverables
Planning	Requirements definition, KRL formula specification, interface design	Specifications document, interface mockups
Design	System architecture, technology selection, component design	Architecture diagram, data schemas
Implementation	Graphical interface coding, KRL algorithm implementation, component integration	Functional source code, operational interface
Testing	Scientific validation of calculations, interface testing, test cases with real data	Test report, results validation
Documentation	Preparation of manuals, technical documentation, user guides	User manual, engineering document

Table 3: Development Cycle Phases, Activities and Deliverables

3. System Analysis and Design

3.1. General Description

The KRL Predictor is a desktop application designed to calculate the Local Refraction Coefficient (KRL), a fundamental parameter in atmospheric and geodetic studies. The system allows users to input meteorological variables and obtain the coefficient value quickly and accurately, with an intuitive visual representation of the result.

3.2. Functional Requirements

- FR-01: The system must allow the entry of four parameters: height (m), temperature (°C), pressure (hPa) and vertical temperature gradient (°C/m).
- FR-02: The system must calculate the KRL using the validated scientific formula.
- FR-03: The system must display the calculation result with 5 decimal places of precision.
- FR-04: The system must provide a visual gauge with a color gradient representing the KRL range (green: low, yellow: normal, red: high).
- FR-05: The system must include informational tooltips for each input field.
- FR-06: The system must allow clearing all fields and restoring the initial state.
- FR-07: The system must validate that input data are valid numeric values.
- FR-08: The system must support interchangeable light and dark modes.
- FR-09: The system must support interchangeable Spanish and English modes.

3.3. Non-Functional Requirements

- NFR-01 (Usability): The interface must be intuitive and require no prior training for users with basic computer knowledge.
- NFR-02 (Portability): The system must be executable on Windows, Linux and macOS without modifications.
- NFR-03 (Performance): The KRL calculation must execute in less than 100ms.
- NFR-04 (Precision): Calculations must maintain double-precision floating-point accuracy.
- NFR-05 (Maintainability): The code must be structured, commented and follow Python PEP 8 standards.

3.4. Use Cases

Actor	User (Scientist, Geodesist, Researcher)
Description	The user inputs atmospheric parameters and obtains the KRL calculation
Preconditions	The application is open and operational
Normal Flow	1. User enters height in meters 2. User enters temperature in °C 3. User enters pressure in hPa 4. User enters vertical gradient (°C/m) 5. User presses the 'Calculate KRL' button 6. System validates the data 7. System executes the KRL calculation 8. System displays the numeric result 9. System updates the visual gauge
Alternative Flow	6a. If data are invalid, the system displays an error message
Postconditions	The KRL is calculated and displayed correctly

Table 4: UC-01: Calculate KRL Coefficient

4. System Architecture

4.1. General Architecture

The system uses a three-layer architecture implemented in a single Python module, optimized for simplicity and maintainability.

Layer	Responsibility	Technology
Presentation	Graphical user interface, event capture, results display	CustomTkinter, Canvas (Tkinter)
Business Logic	KRL formula implementation, data validation, flow control	Python (class methods)
Visualization	Visual gauge rendering, color gradient, position indicator	Canvas (Tkinter), 2D graphics

Table 5: Responsibility Distribution

4.2. Technology Stack

Component	Technology / Library
Programming Language	Python 3.8+
GUI Framework	CustomTkinter (modern interface over Tkinter)
Additional UI Components	CTkToolTip (informational tooltips)
Mathematical Calculations	Python standard math library

Table 6: Technology Stack

4.3. System Components

Main Class: App

The App class inherits from ctk.CTk and encapsulates all application logic. Its main methods are:

- __init__(): Constructor that initializes the graphical interface
- toggle_mode(): Switches between light and dark mode
- calcular(): Executes the KRL calculation according to the scientific formula
- dibujar_gradiente(): Renders the visual color gradient
- actualizar_medidor(): Updates the indicator position on the gauge
- limpiar(): Resets all fields and the visual state

5. Implementation Details

5.1. KRL Coefficient Formula

The Local Refraction Coefficient calculation is implemented using the following scientific formula:

$$KL = 121 \times (P \times 1.3332) / T^2 \times (0.0343 + \partial T/\partial Z)$$

Where:

- KL: Local Refraction Coefficient
- P: Atmospheric pressure in hectopascals (hPa)
- T: Absolute temperature in Kelvin (K)
- ∂T/∂Z: Vertical temperature gradient (°C/m)

5.2. Data Validation

The system implements validation using a try-except block that catches numeric conversion errors (ValueError) and displays informational messages to the user in the event of invalid data.

5.3. Visual Gauge

The visual gauge uses a dynamically generated color gradient ranging from green (low KRL values) to yellow (normal values) and red (high values). The established ranges are:

- Green (Low): KRL ≤ 0.11
- Yellow (Normal): 0.11 < KRL ≤ 0.20
- Red (High): KRL > 0.20

6. Testing and Validation

6.1. Testing Strategy

A testing strategy was implemented based on:

- Unit Testing: Validation of the mathematical formula with known values
- Interface Testing: Verification of controls and visualization behavior

- Scientific Validation Testing: Comparison of results with manual calculations by the domain expert
- Usability Testing: Evaluation of user experience

6.2. Test Cases

ID	Description	Input	Expected Result
TC-01	Calculation with normal values	H=100, T=20, P=1013, G=0.01	KRL calculated with 5 decimal places
TC-02	Non-numeric input validation	H='abc', T=20, P=1013, G=0.01	Error message displayed
TC-03	Clear function	Click on 'Clear' button	All fields empty, gauge reset
TC-04	Light/dark mode toggle	Toggle switch	Theme changes correctly

Table 7: Test Cases

7. Installation and Deployment

7.1. System Requirements

- Operating System: Windows 10+, Linux (Ubuntu 18.04+), macOS 10.14+
- Python: Version 3.8 or higher
- RAM: Minimum 512 MB
- Disk Space: 50 MB

7.2. Installation Process

No installation required, as the system is distributed as a standalone executable file.

8. Maintenance and Support

Maintenance Plan

- Corrective Maintenance: Correction of errors reported by users
- Adaptive Maintenance: Updates due to changes in dependencies or operating systems
- Perfective Maintenance: Interface improvements and performance optimizations

9. Conclusions

The KRL Predictor system has been successfully developed using a simplified agile methodology, appropriate for the context of a small team. The implementation fulfills the scientific objectives established by the domain expert, providing a precise and efficient tool for the calculation of the Local Refraction Coefficient.

The use of cross-platform technologies ensures the system is accessible across diverse work environments, while the intuitive graphical interface facilitates adoption by users without advanced technical expertise.

The modular architecture and well-documented code ensure the future maintainability of the system, allowing for improvements and adaptations to be incorporated as needs arise within the scientific community.